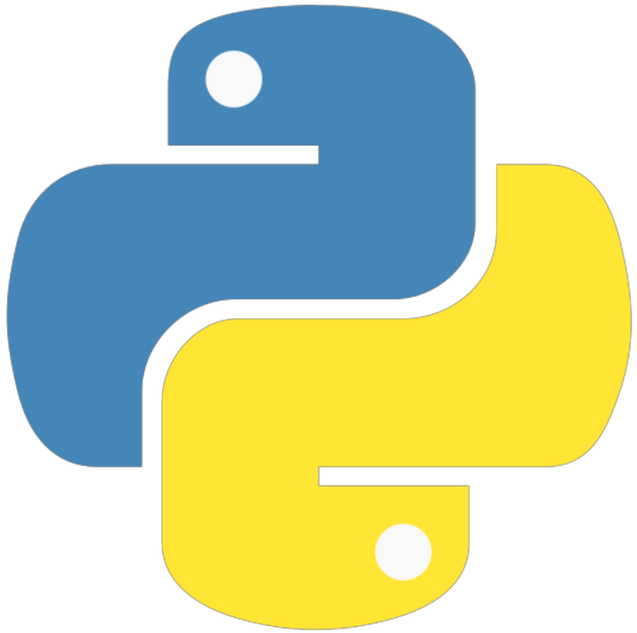
UNIVERSITÄT OSNABRÜCK

Programmieren für PsychologInnen

Plotting in Python



Allgemeine Psychologie I
B.Sc. Psychologie / M.Sc. Psychologie - Seminar Programmieren
06.06.2025

Merle Sagehorn, M.Sc.

Today

- Aufgaben / Mini-Projekte

- Plotting
  - How To Plot
  - Arten von Plots
  - Subplots

Nachtrag: Numpy Arrays

```
array_1 = np.random.rand(1,4)
array_2 = np.random.randn(1,4)
array_3 = np.random.randn(7)
array_4 = np.random.randint(20, size=5)
array_5 = np.random.permutation(array_1)
```

np.random.rand(1,4) erzeugt einen **2**-dimensionalen Array, auch wenn die erste Dimension die Größe 1 hat.

```
[[0.38551116 0.39718666 0.75151531 0.68622034]]
[[0.4142128  1.28851495 1.92365961 0.8830765 ]]
[ 1.50699604 -0.65707693 -0.81206418  0.34174937  0.50183117 -0.57402249
 -0.95936834]
[ 1  1 12  7  9]
[[0.38551116 0.39718666 0.75151531 0.68622034]]
1.1273659630440827
7.0
0.5541473950305669
TtestResult(statistic=array([nan, nan, nan, nan]), pvalue=array([nan, nan, nan, nan]), df=array([0, 0, 0, 0]))
[[1. 1.]
 [1. 1.]]
[[1.        0.5996012]
 [0.5996012 1.       ]]
```

2-dimensionaler Array → [[…]]

t-Test nicht auf **2**-dimensionalen Arrays anwendbar → ungültiges Ergebnis

Nachtrag: Numpy Arrays

```
array_1 = np.random.rand(4)
array_2 = np.random.randn(4)
array_3 = np.random.randn(7)
array_4 = np.random.randint(20, size=5)
array_5 = np.random.permutation(array_1)
```

np.random.rand(4) erzeugt einen **1**-dimensionalen Array.

```
[0.08854631 0.75505128 0.95241476 0.09148484]
[-0.14616933  0.35343139  0.18562029  1.64258056]
[ 0.3393381   0.31015605  0.68422712 -0.87962639  2.29293906 -0.37064242
  1.09260004]
[19  4 12  5 11]
[0.95241476 0.08854631 0.75505128 0.09148484]
0.5088657287039646
11.0
0.6787907803268989
TtestResult(statistic=0.07158006500682652, pvalue=0.9474409424130401, df=3)
[[ 1.         -0.01987596]
 [-0.01987596  1.        ]]
[[ 1.         -0.36007094]
 [-0.36007094  1.        ]]
```

1-dimensionaler Array → […]

t-Test funktioniert

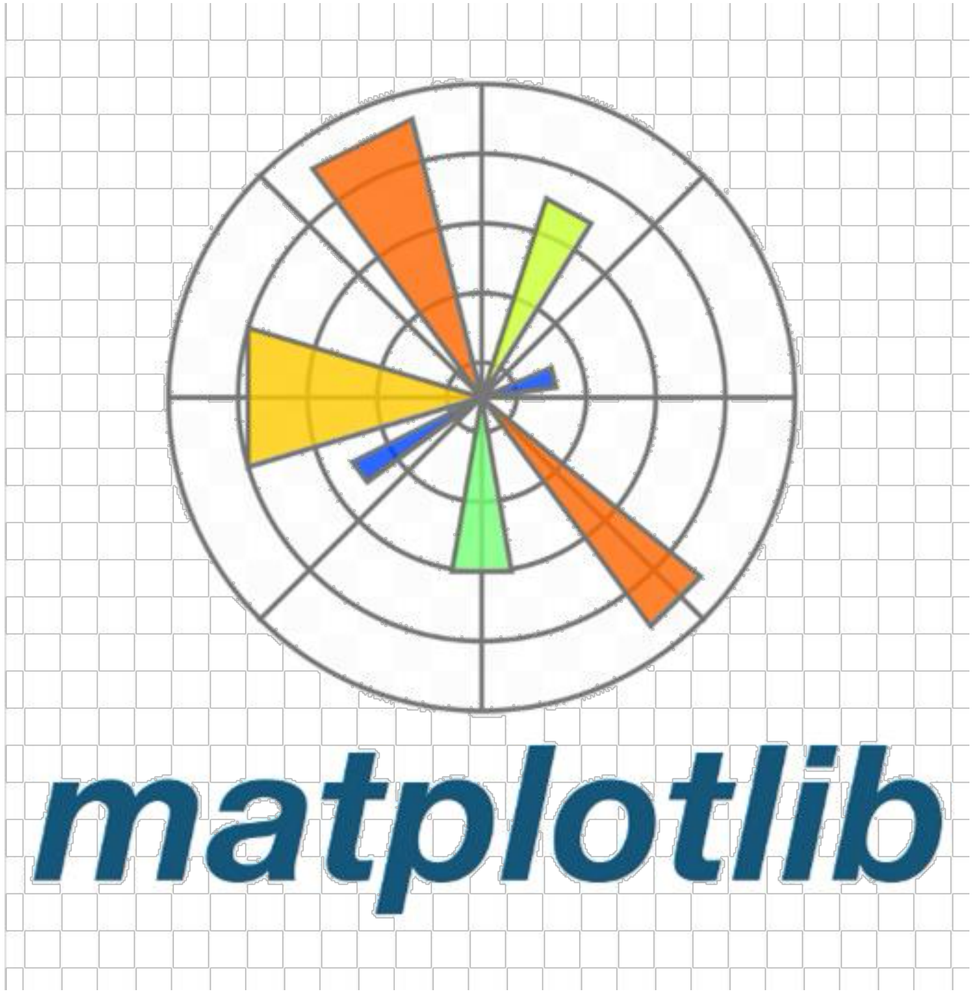
Eine neue Bibliothek

- Matplotlib ist eine umfangreiche Bibliothek zur Erstellung von 2D- und 3D-Diagrammen in Python. Sie wird häufig für die Datenvisualisierung verwendet.

- Hohe Integrationsfähigkeit mit anderen Bibliotheken wie NumPy, pandas und SciPy

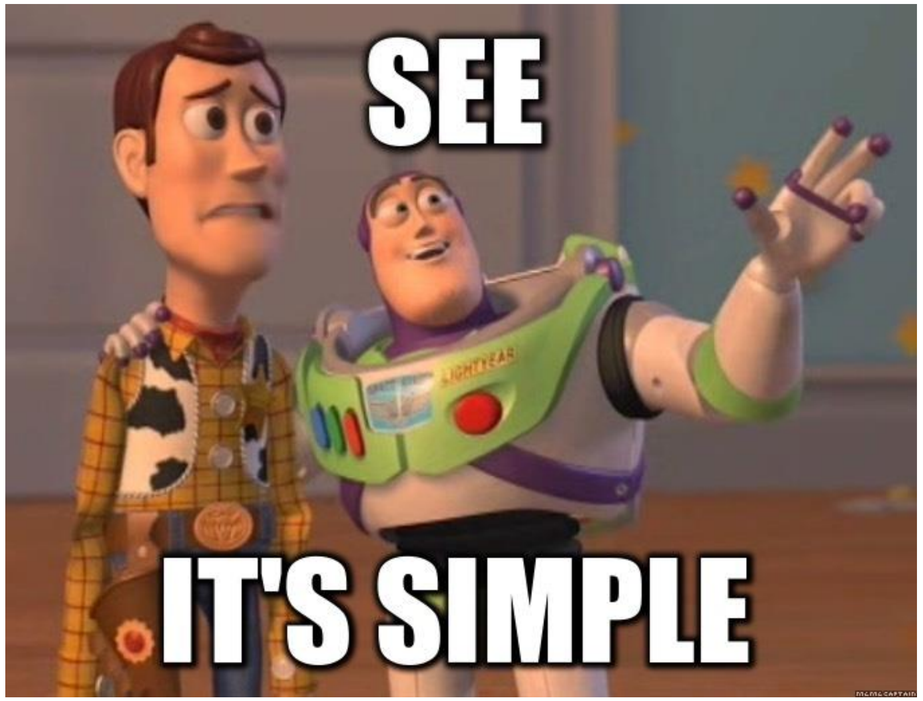
- Besonders nützlich für wissenschaftliche und technische Visualisierungen, einschließlich der Anpassung von Achsen, Beschriftungen und Legenden.

- ***Am wichtigsten***: Sie ist gut dokumentiert!



5 steps to a pretty plot!

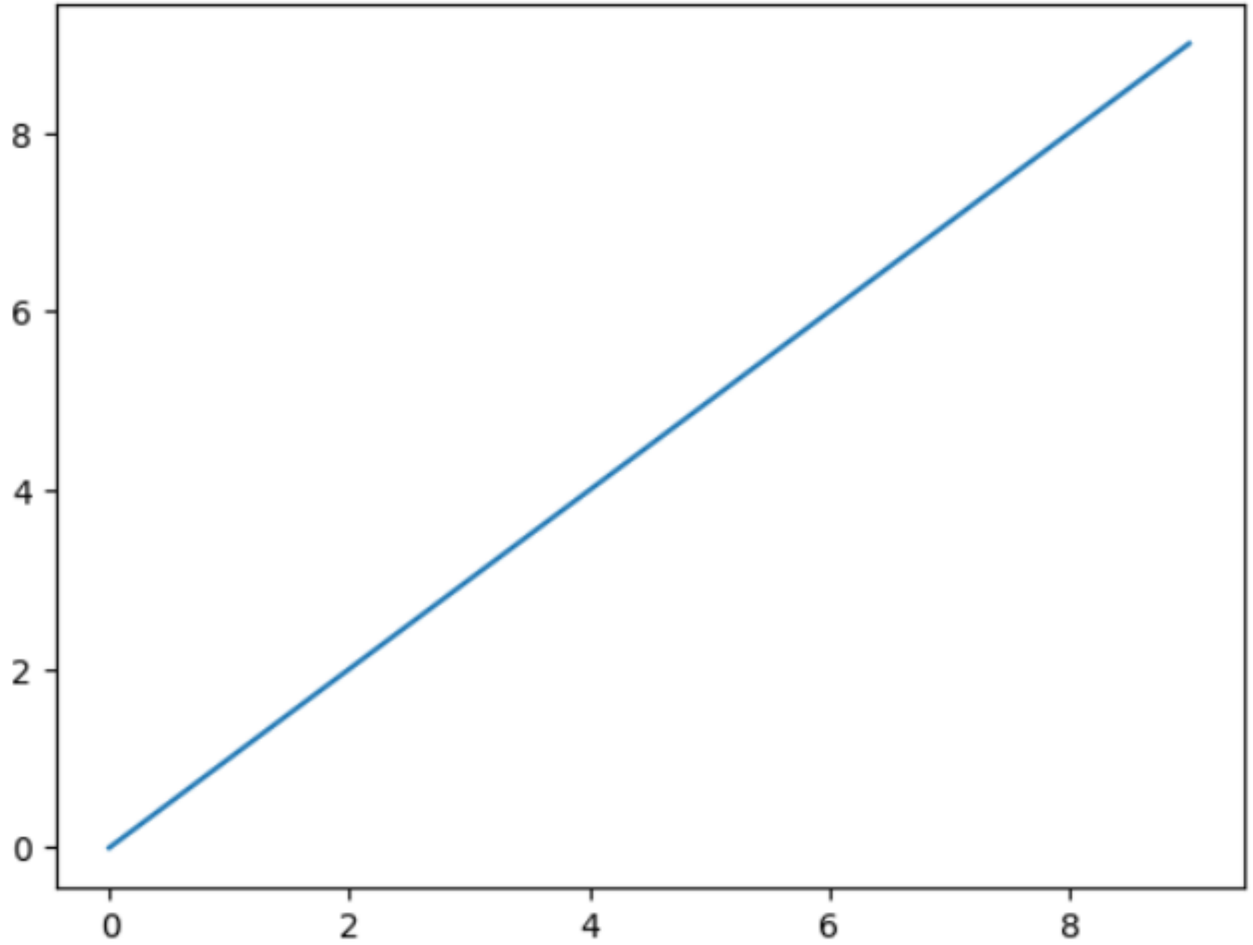
1. Entscheide dich für eine Art von Plot.
2. Kreiere Daten für deinen Plot.
3. Gib den plot Befehl ein.
4. … füge ein paar Extras hinzu.
5. Sei stolz.



See, it *is* that simple!

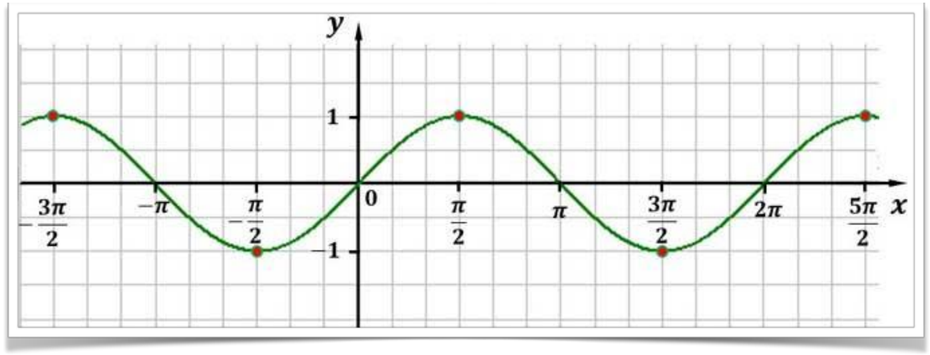
```
import numpy as np
import matplotlib.pyplot as plt
```

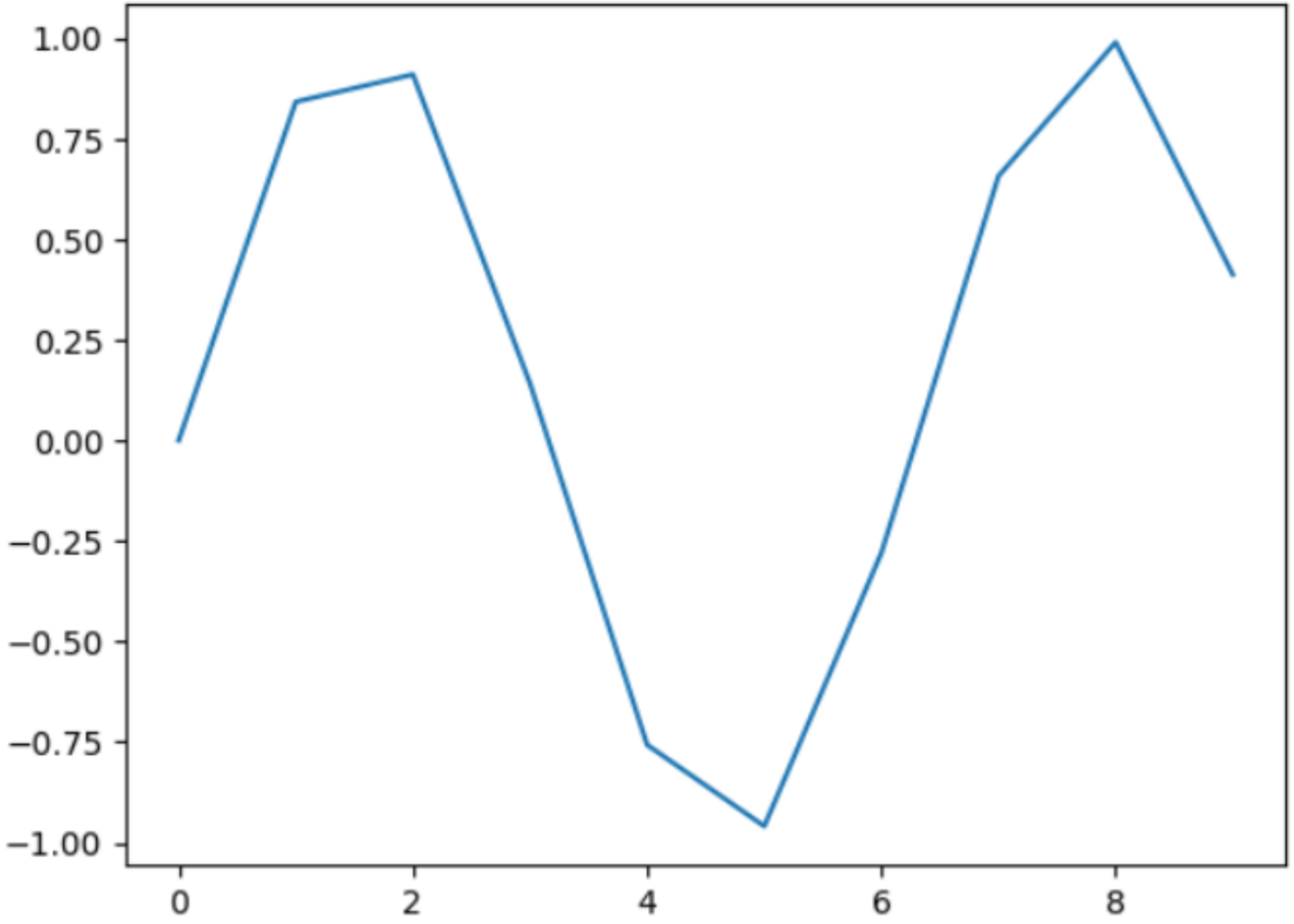
```
# create data
x = np.arange(10)
y = np.arange(10)

# plot it
plt.plot(x,y)

plt.show()
```



Or is it?

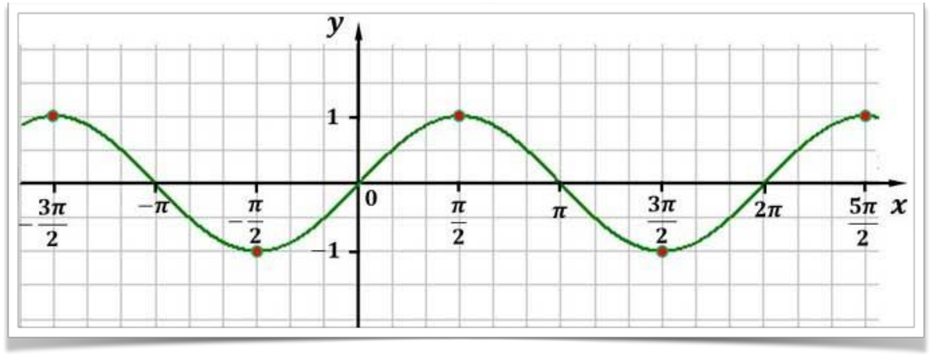
```
# Daten erstellen
x = np.arange(10)
y = np.sin(x)

# Plot erstellen
plt.plot(x, y)
```

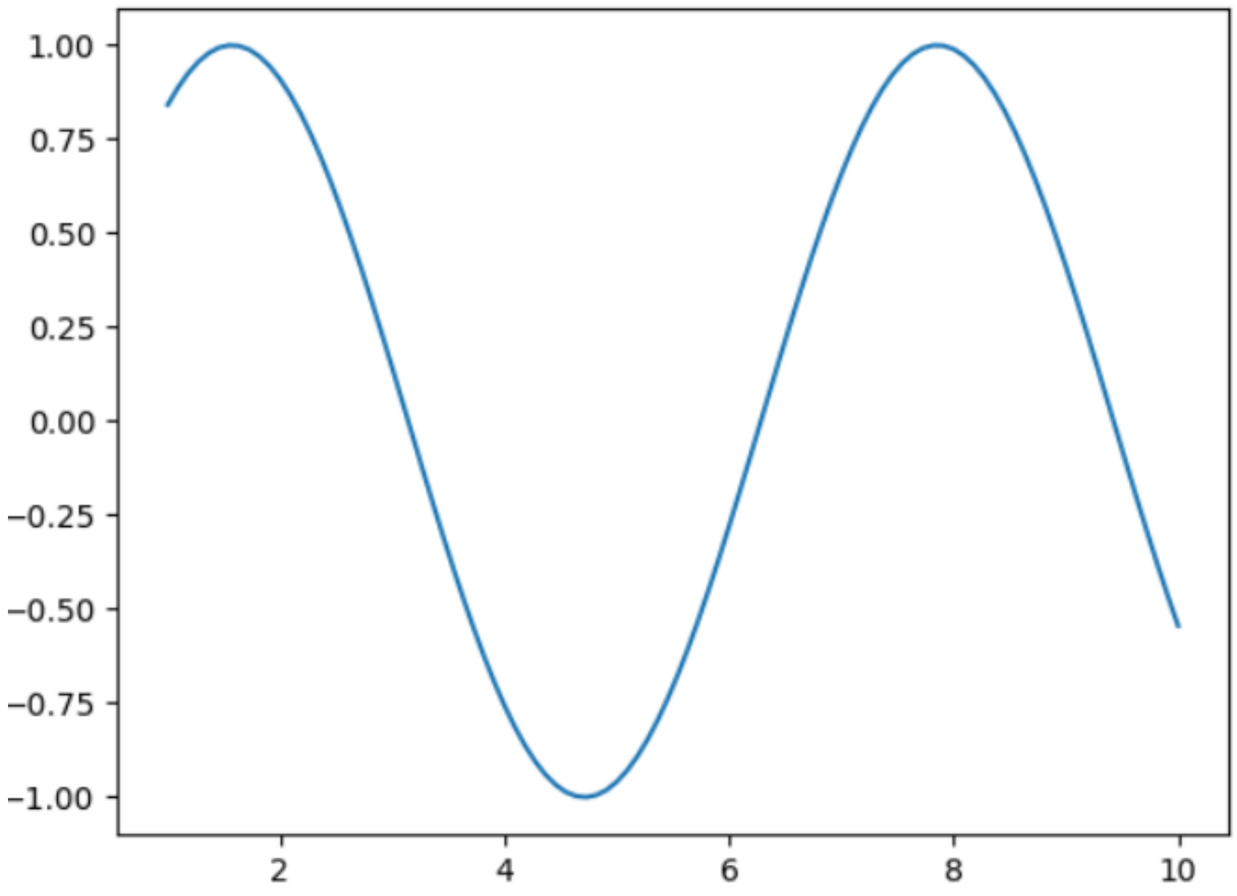




It certainly is…

```
# Daten erstellen
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Plot erstellen
plt.plot(x, y)
```



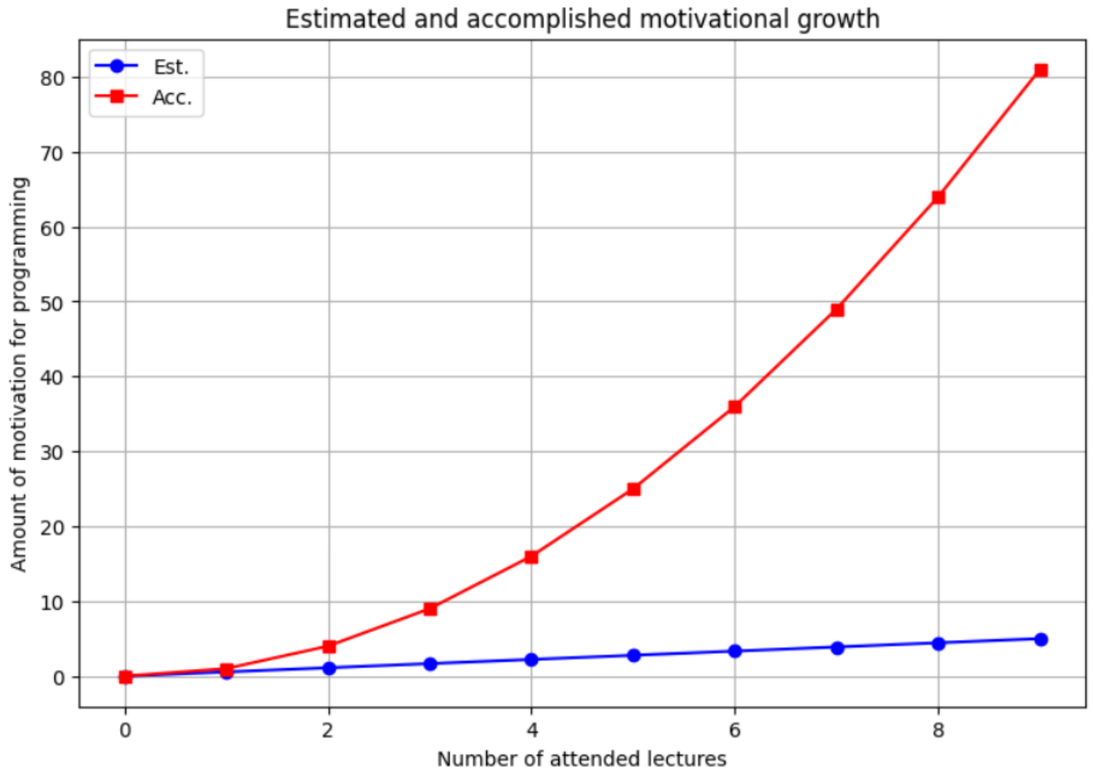


Plotting Basics

- Es braucht Daten.
  - z.B.: Reaktionszeiten, Test-Scores, Performance-Verbesserung, EEG-Daten, …
- Es braucht eine passende x-Achse.
  - z.B.: Gruppen, Art des Tests, Zeitpunkte, kontinuierliche Zeit, …
- Ein paar nützliche Zusatz-Befehle, um den Plot ganz nett aussehen zu lassen.
  - Farbe, Größe, Beschriftung, …

Plotting - Beispiel

```
x = np.arange(10)
y1 = np.linspace(0, 5, 10)
y2 = np.power(x, 2)


# Plot erstellen
plt.figure(figsize=(9, 6))  # Größe des Plots festlegen
plt.plot(x, y1, label='Est.', color='blue', marker='o')  # Erste Kurve
plt.plot(x, y2, label='Acc.', color='red', marker='s')  # Zweite Kurve

# Titel und Achsenbeschriftungen hinzufügen
plt.title('Estimated and accomplished motivational growth')
plt.xlabel('Number of attended lectures')
plt.ylabel('Amount of motivation for programming')

# Raster hinzufügen
plt.grid()

# Legende hinzufügen
plt.legend()
```



Plot Colors & Markers

black	bisque	forestgreen	slategrey
dimgray	darkorange	limegreen	lightsteelblue
dimgrey	burlywood	darkgreen	cornflowerblue
gray	antiquewhite	green	royalblue
grey	tan	lime	ghostwhite
darkgray	navajowhite	seagreen	lavender
darkgrey	blanchedalmond	mediumseagreen	midnightblue
silver	papayawhip	springgreen	navy
lightgray	moccasin	mintcream	darkblue
lightgrey	orange	mediumspringgreen	mediumblue
gainsboro	wheat	mediumaquamarine	blue
whitesmoke	oldlace	aquamarine	slateblue
white	floralwhite	turquoise	darkslateblue
snow	darkgoldenrod	lightseagreen	mediumslateblue
rosybrown	goldenrod	mediumturquoise	mediumpurple
lightcoral	cornsilk	azure	rebeccapurple
indianred	gold	lightcyan	blueviolet
brown	lemonchiffon	paleturquoise	indigo
firebrick	khaki	darkslategray	darkorchid
maroon	palegoldenrod	darkslategrey	darkviolet
darkred	darkkhaki	teal	mediumorchid
red	ivory	darkcyan	thistle
mistyrose	beige	aqua	plum
salmon	lightyellow	cyan	violet
tomato	lightgoldenrodyellow	darkturquoise	purple
darksalmon	olive	cadetblue	darkmagenta
coral	yellow	powderblue	fuchsia
orangered	olivedrab	lightblue	magenta
lightsalmon	yellowgreen	deepskyblue	orchid
sienna	darkolivegreen	skyblue	mediumvioletred
seashell	greenyellow	lightskyblue	deeppink
chocolate	chartreuse	steelblue	hotpink
saddlebrown	lawngreen	aliceblue	lavenderblush
sandybrown	honeydew	dodgerblue	palevioletred
peachpuff	darkseagreen	lightslategray	crimson
peru	palegreen	lightslategrey	pink
linen	lightgreen	slategray	lightpink

‘color‘ (Farbe der Linie)

‘**marker**‘ (Form der Datenpunkt-Markierung)
o = Kreis; s = square; …

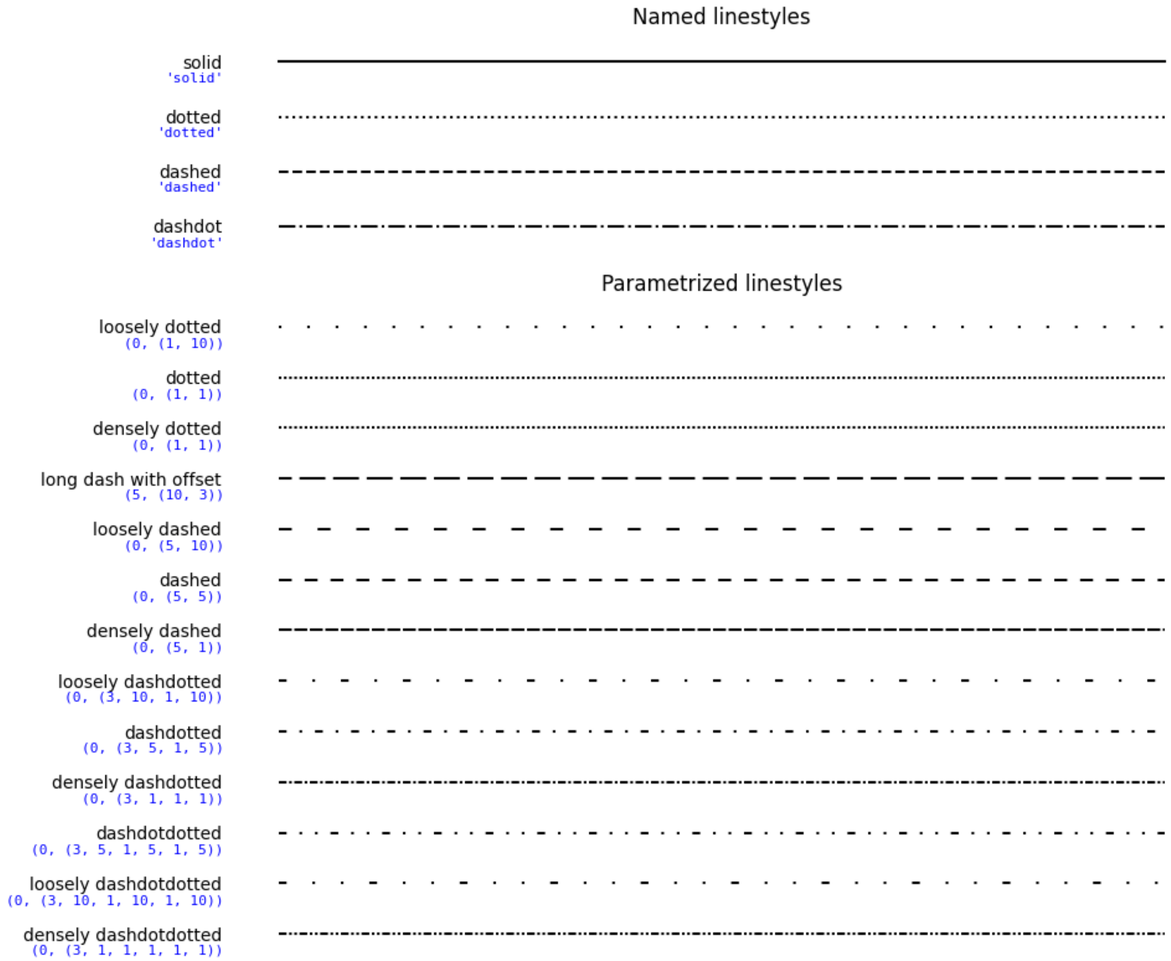
‘**markerfacecolor**‘ (Farbe des Markers)

‘**makersize**‘ (Größe der Marker)

Plot Line Specifics

„LineStyle“
Linienstile

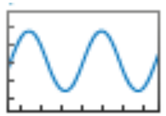
„LineWidth“
Linienstärke



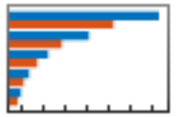
Plot Arten

Plot Arten

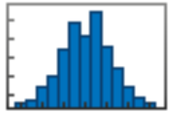
- 2D-Lineplot → plot



- Barchart → bar, barh

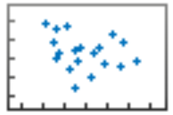


- Histogramm → hist

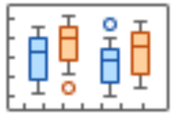


- Pie chart → pie

- Scatterplot → scatter

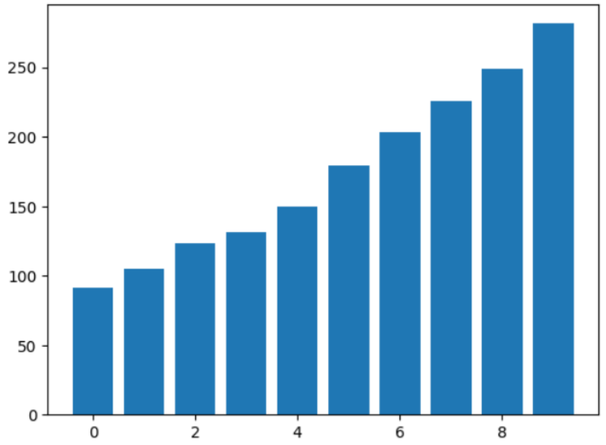


- Boxplot → boxplot

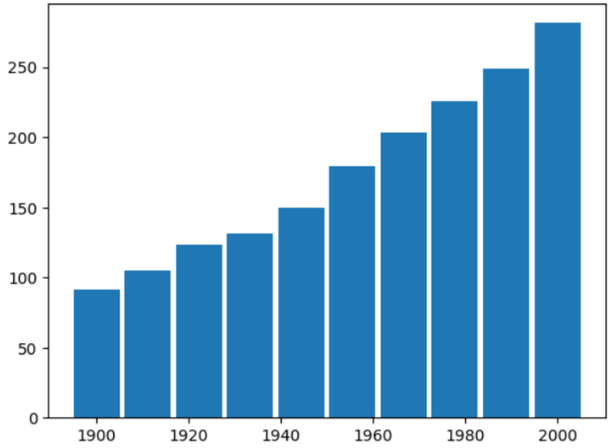


Barcharts

```
y = np.array([91, 105, 123.5, 131, 150, 179, 203, 226, 249, 281.5])
x = np.arange(10)
plt.bar(x, y)
```

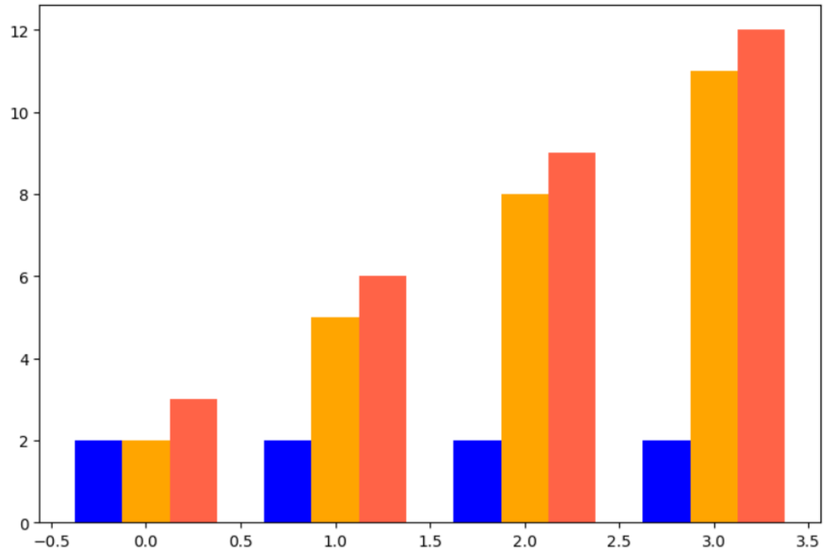



```
y = np.array([91, 105, 123.5, 131, 150, 179, 203, 226, 249, 281.5])
x = np.linspace(1900, 2000, 10)
plt.bar(x, y, width = 10)
```



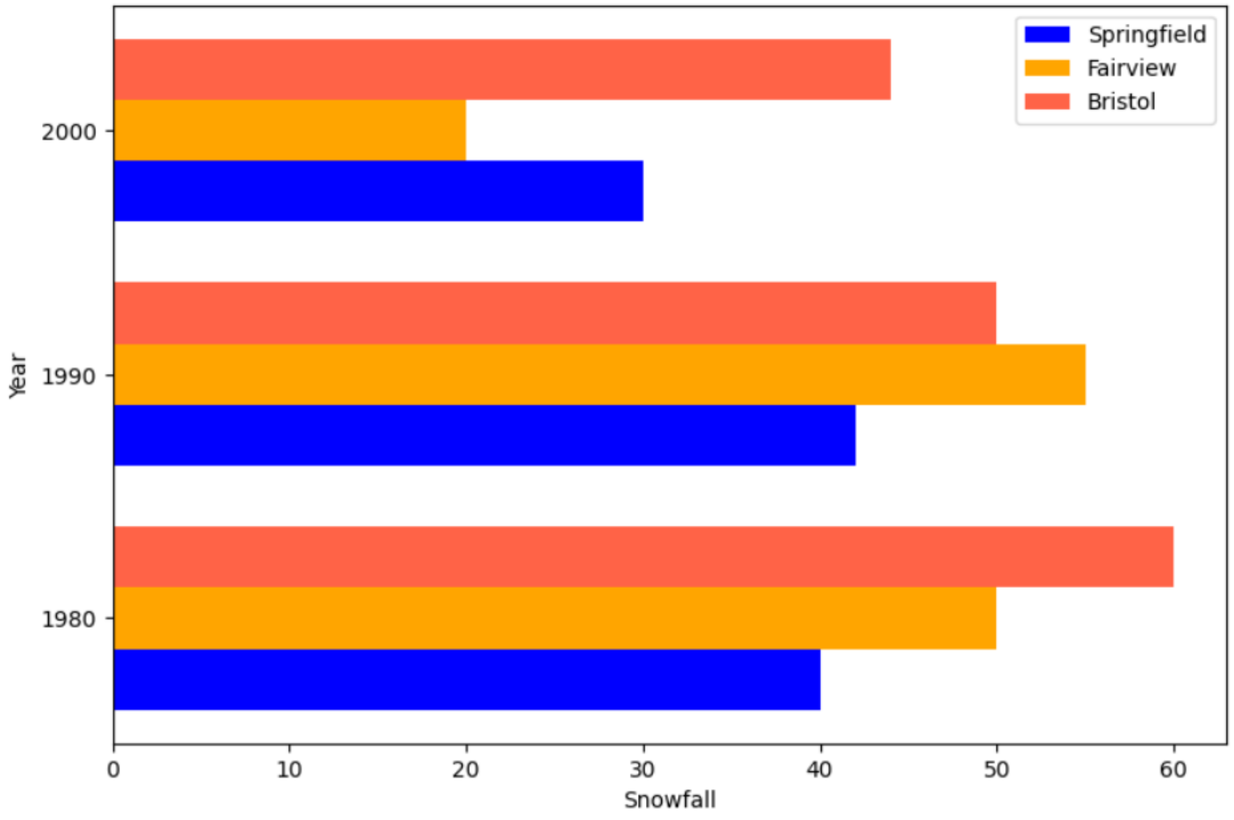
Barcharts

```
x = np.arange(4)
y = np.array([[2, 2, 3], [2, 5, 6], [2, 8, 9], [2, 11, 12]])

width = 0.25  # Die Breite der Balken

plt.figure(figsize=(9, 6))
plt.bar(x - width, y[:, 0], width, color='blue')
plt.bar(x, y[:, 1], width, color='orange')
plt.bar(x + width, y[:, 2], width, color='tomato')
```



Barcharts

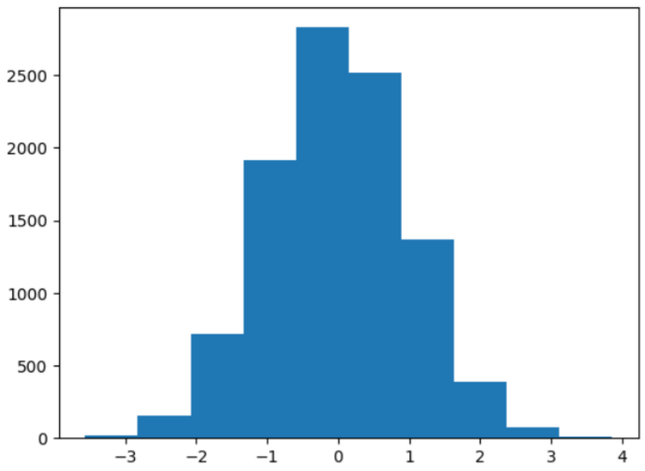
```
x = np.arange(3)
y = np.array([[40, 50, 60], [42, 55, 50], [30, 20, 44]])

width = 0.25

plt.figure(figsize=(9, 6))
plt.barh(x - width, y[:, 0], width, label = 'Springfield', color = 'blue')
plt.barh(x, y[:, 1], width, label = 'Fairview', color = 'orange')
plt.barh(x + width, y[:, 2], width, label = 'Bristol', color = 'tomato')

plt.xlabel("Snowfall")
plt.ylabel("Year")
plt.yticks(x, ["1980", "1990", "2000"])

plt.legend()
```



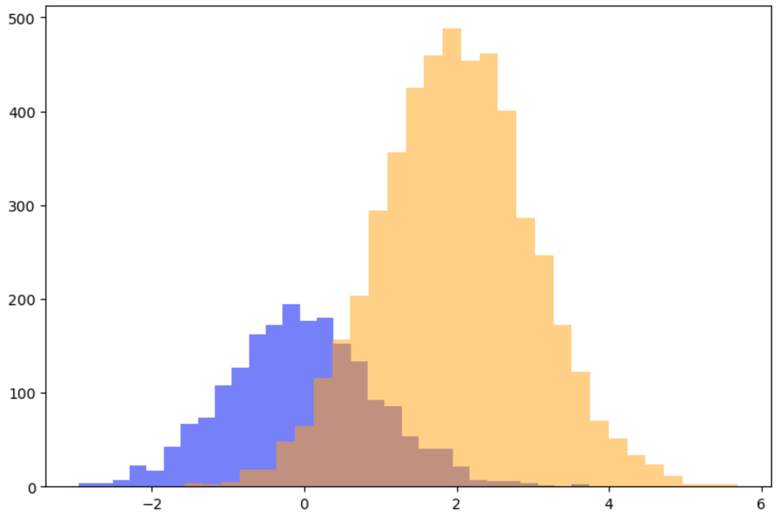
Histogramm

Histogramme sind eine Art von Balkendiagrammen für numerische Daten, die die Daten in Bins gruppieren.

```
x = np.random.randn(10000, 1)
plt.hist(x)
```

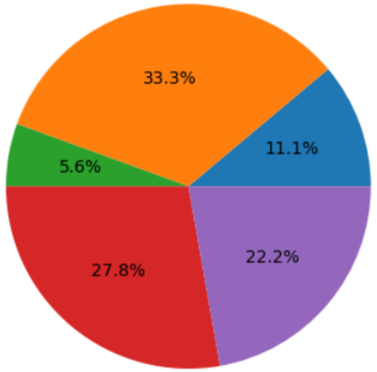



```
x = np.random.randn(2000, 1)
y = 2 + np.random.randn(5000, 1)

plt.figure(figsize = (9,6))
plt.hist(x, bins = 30, alpha = 0.5, color = 'blue')
plt.hist(y, bins = 30, alpha = 0.5, color = 'orange')
```



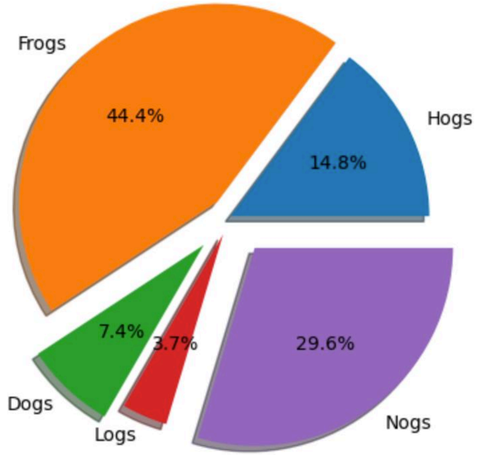
Pie Charts

```
x = np.array([1, 3, 0.5, 0.25, 2])
plt.pie(x)
```

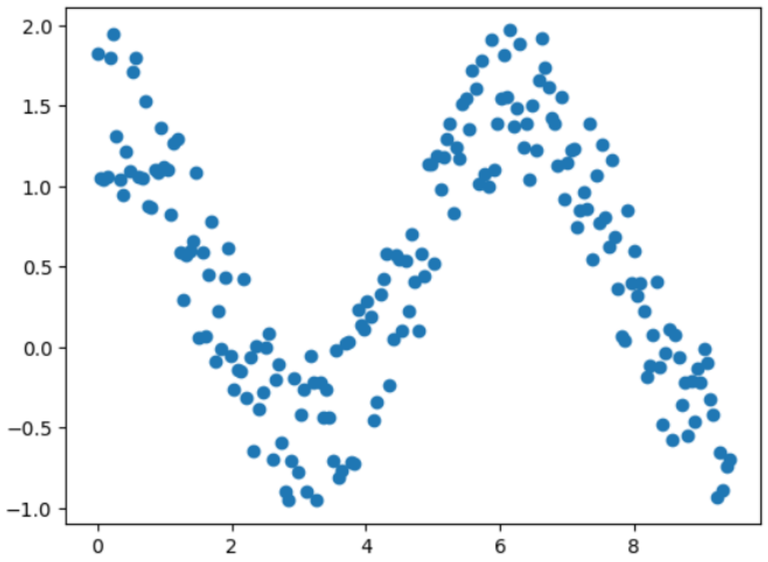



```
x = np.array([1, 3, 0.5, 0.25, 2])
plt.pie(x, autopct = "%1.1f%%")
```

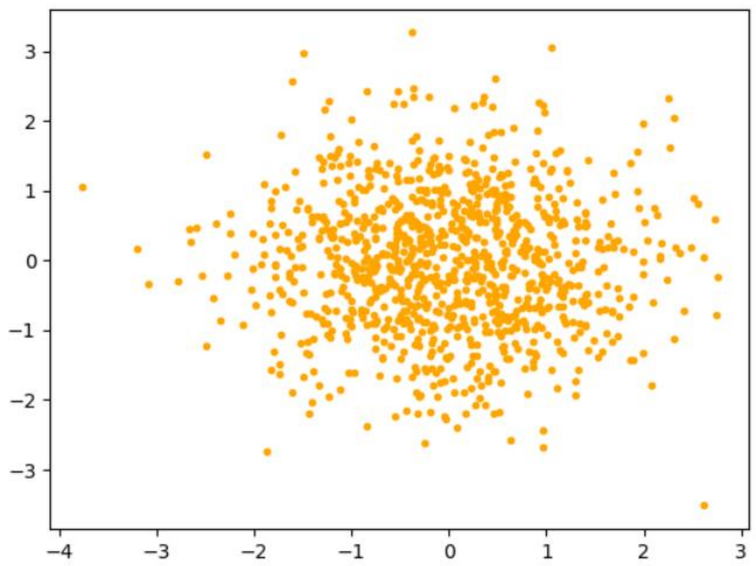
```
x = np.array([1, 3, 0.5, 0.25, 2])
explode = np.array([0, 0.1, 0.2, 0.1, 0.2])
labels = np.array(["Hogs", "Frogs", "Dogs", "Logs", "Nogs"])

plt.pie(x, autopct = "%1.1f%%",
        shadow = True,
        explode = explode,
        labels = labels
```



Scatter Plot



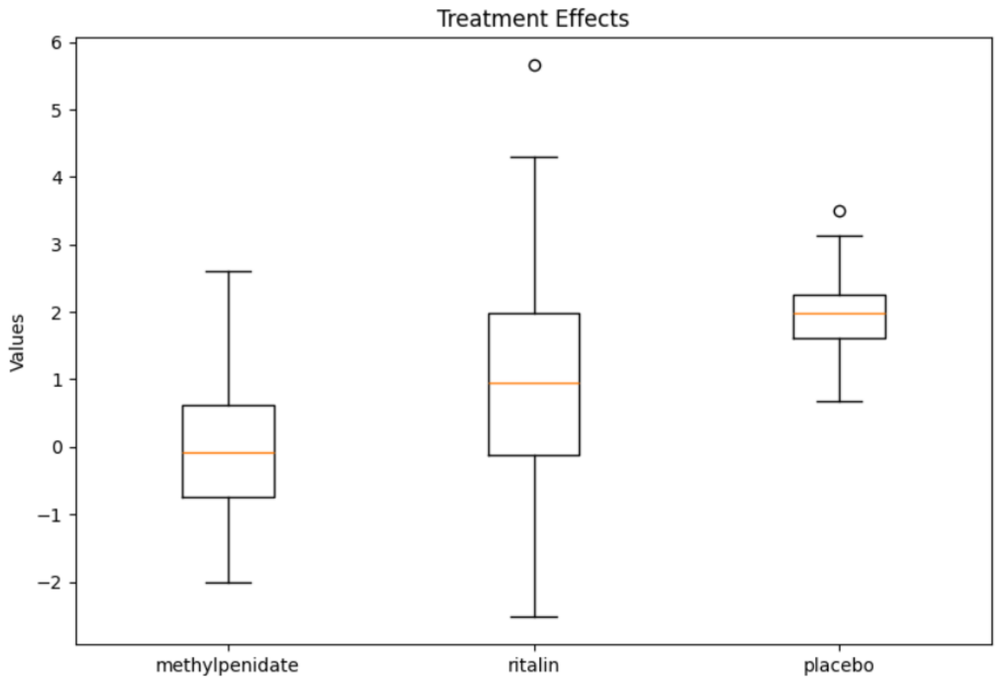

```
x = np.linspace(0, 3*np.pi, 200)
y = np.cos(x) + np.random.rand(1,200)

plt.scatter(x, y)
```




```
x = np.random.randn(1000, 1)
y = np.random.randn(1000, 1)

plt.scatter(x, y, marker = '.', color = 'orange')
```

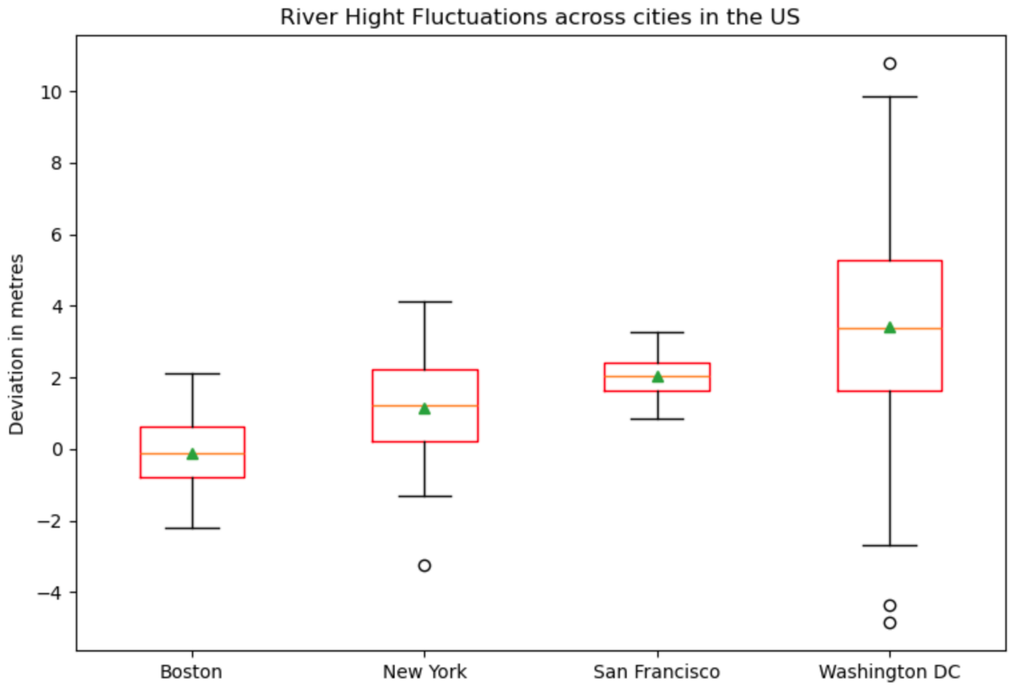
Boxplot

```
data1 = np.random.normal(loc = 0, scale = 1, size = 100)    # Gruppe 1
data2 = np.random.normal(loc = 1, scale = 1.5, size = 100)  # Gruppe 2
data3 = np.random.normal(loc = 2, scale = 0.5, size = 100)  # Gruppe 3

data = [data1, data2, data3]

plt.figure(figsize = (9,6))
plt.boxplot(data, labels = ["methylpenidate", "ritalin", "placebo"])

plt.title("Treatment effects")
plt.ylabel("Values")
plt.show();
```



Boxplot

```
data1 = np.random.normal(loc = 0, scale = 1, size = 100)
data2 = np.random.normal(loc = 1, scale = 1.5, size = 100)
data3 = np.random.normal(loc = 2, scale = 0.5, size = 100)
data4 = np.random.normal(loc = 3, scale = 3, size = 100)

data = [data1, data2, data3, data4]

plt.figure(figsize = (9,6))
plt.boxplot(data,
            boxprops = dict(color = "red"),
            labels = ["Boston", "New York", "San Francisco", "Washington DC"],
            showmeans = True)

plt.title("River Hight Fluctuations across cities in the US")
plt.ylabel("Deviation in metres")
plt.show();
```



Plot Styles

Matplotlib stellt verschieden Plot-Styles zur Verfügung, die ihre eigenen Custom Colours und Schriftarten etc. mitbringen.

Mithilfe von

```
plt.style.available
```

kann man sich alle verfügbaren Styles ausgeben lassen:

```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirty
eight', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-p
alette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-
pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorb
lind10']
```

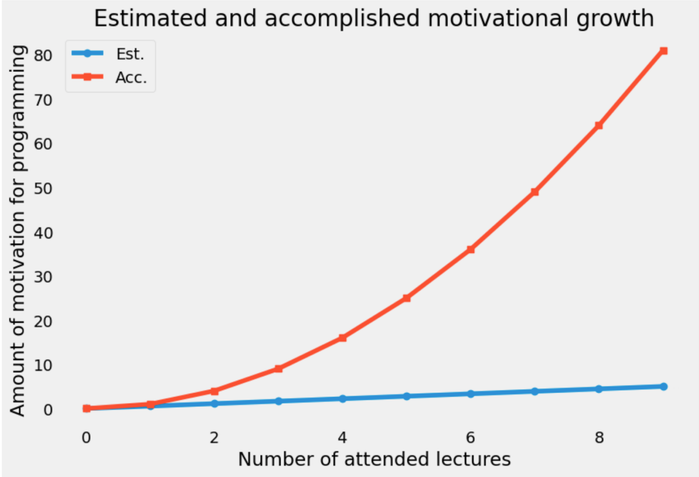
Und dann mithilfe von

```
plt.style.use('')
```

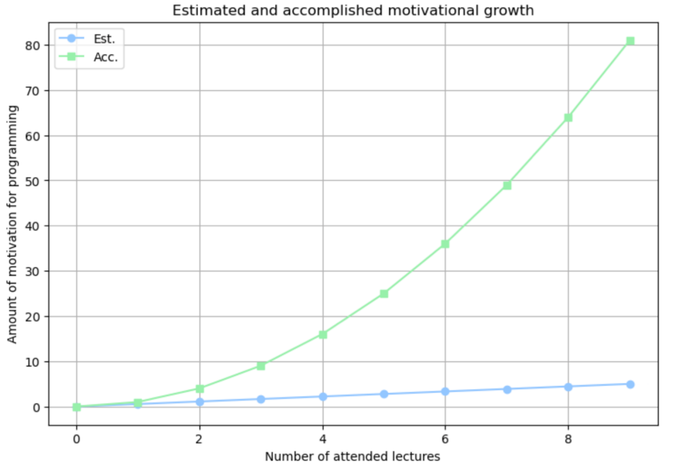
verwenden.

Plot Styles

```
plt.style.use('fivethirtyeight')
```

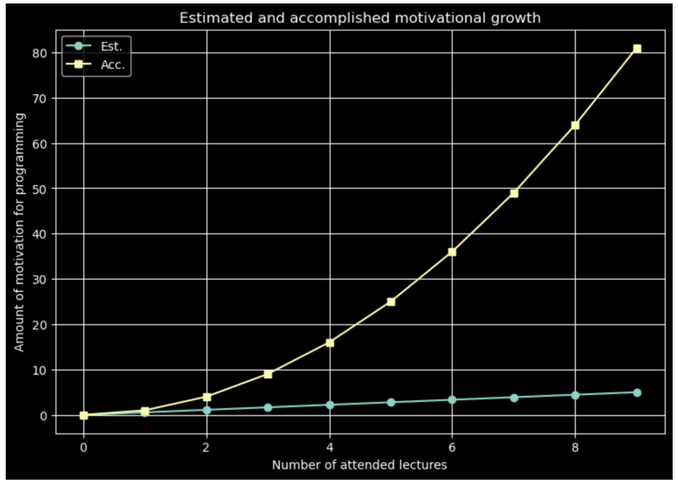



```
plt.style.use('seaborn-v0_8-pastel')
```




```
plt.style.use('dark_background')
```

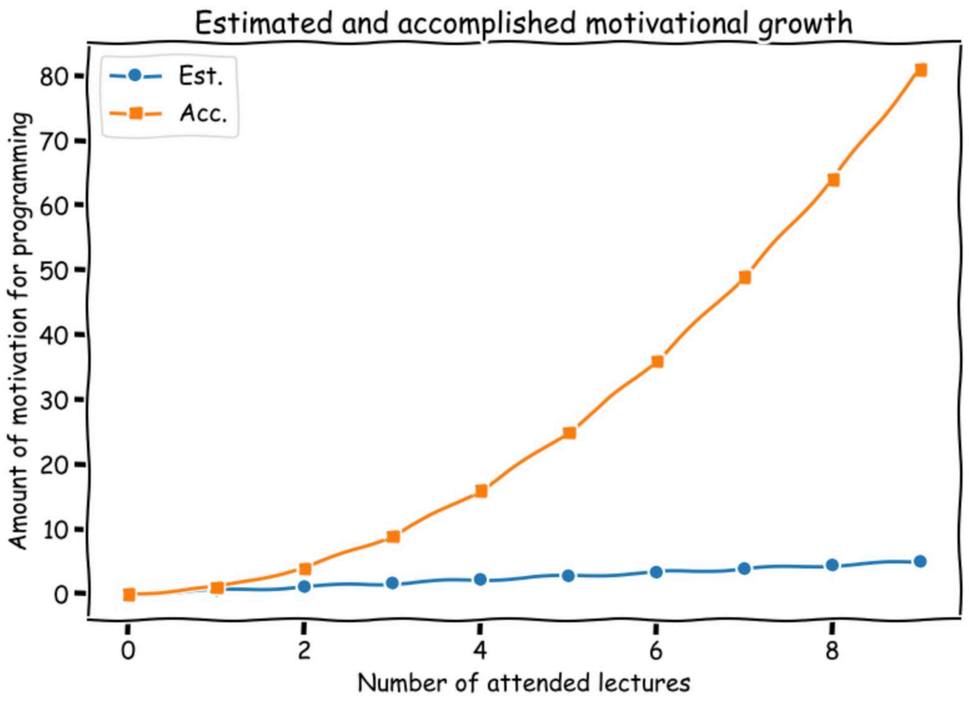
↳ überschreibt alles andere



Plot Styles

```python
import logging
# Bei der verwendung von "xkcd" werden häufig jede Menge Warnungen ausgespuckt.
# Die kommen nur zustande, da der Matplotlib Fontmanager nach Schriftarten sucht,
# die er nicht finden kann. Das ist aber nicht weiter wichtig und kann ignoriert werden.
# Um diese zu unterdrücken, kann man folgenden Befehl verwenden:
logging.getLogger('matplotlib.font_manager').setLevel(logging.ERROR)
# Setze das Logging für Matplotlib Font Manager auf ERROR
```

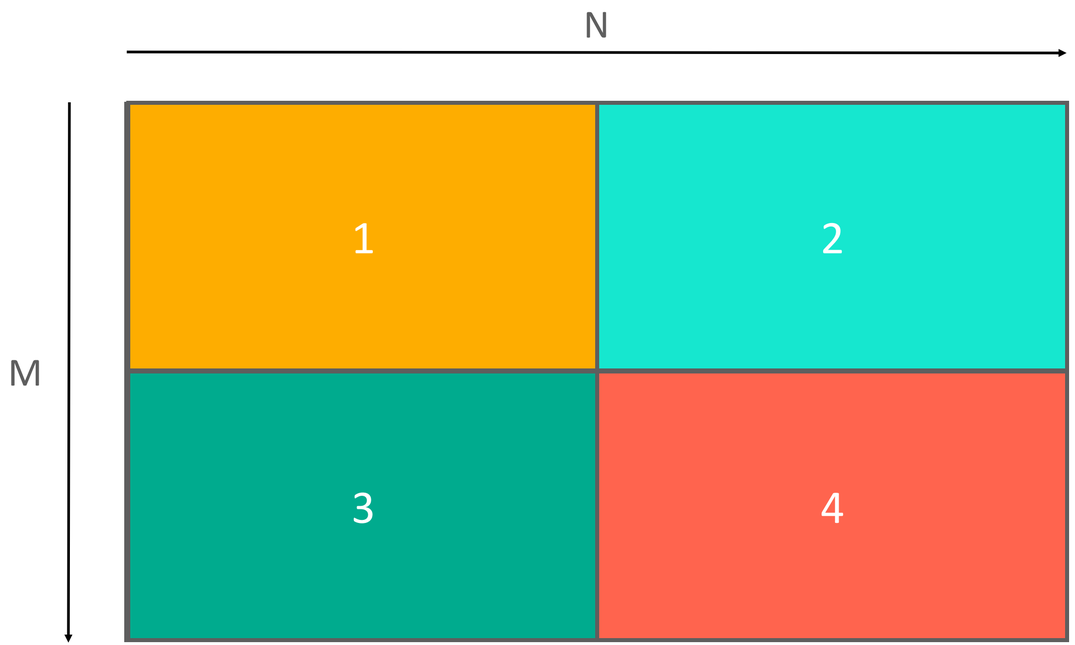
```python
plt.xkcd()
```



Subplot

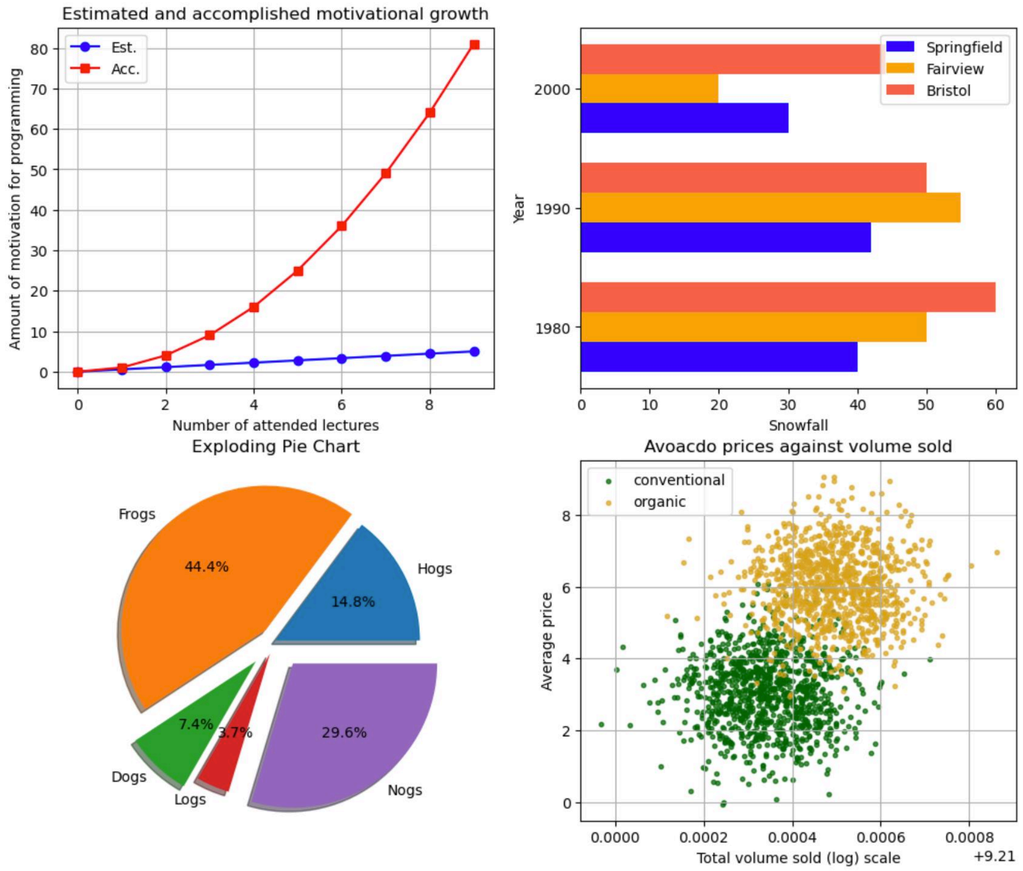
- Eine Figure in kleinere Bereiche unterteilen, um mehrere Plots neben- und übereinander darzustellen.

```
fig, axs = plt.subplots(M, N)  ⟶  fig, axs = plt.subplots(2, 2)
```



Subplot

```python
## Subplot
fig, axs = plt.subplots(2, 2, figsize = (12,10))

# Plot 1:
axs[0, 0].plot(x, y_exp, label = 'Est.', color = 'blue', marker = 'o')
axs[0, 0].plot(x, y_acc, label = 'Acc.', color = 'red', marker = 's')
axs[0, 0].set_title('Estimated and accomplished motivational growth')
axs[0, 0].set_xlabel('Number of attended lectures')
axs[0, 0].set_ylabel('Amount of motivation for programming')
axs[0, 0].grid()
axs[0, 0].legend()

# Plot 2
axs[0, 1].barh(years - width, snow[:,0], width, label = 'Springfield',  color = 'blue')
axs[0, 1].barh(years , snow[:,1], width, label = 'Fairview', color = 'orange')
axs[0, 1].barh(years + width, snow[:,2], width, label = 'Bristol', color = 'tomato')
axs[0, 1].set_xlabel("Snowfall")
axs[0, 1].set_ylabel("Year")
axs[0, 1].set_yticks(years, ["1980", "1990", "2000"])
axs[0, 1].legend()

# Plot 3
axs[1, 0].pie(freq, autopct = "%1.1f%%", shadow = True, explode = explode, labels = labels)
axs[1, 0].set_title("Exploding Pie Chart")

# Plot 4
axs[1, 1].scatter(np.log(x1) ,y1 , marker = '.', color = 'darkgreen', label = 'conventional', alpha = .7)
axs[1, 1].scatter(np.log(x2) ,y2 , marker = '.', color = 'goldenrod', label = 'organic', alpha = .7)
axs[1, 1].set_title("Avoacdo prices against volume sold")
axs[1, 1].set_xlabel("Total volume sold (log) scale")
axs[1, 1].set_ylabel("Average price")
axs[1, 1].grid()
axs[1, 1].legend()

plt.show();
```

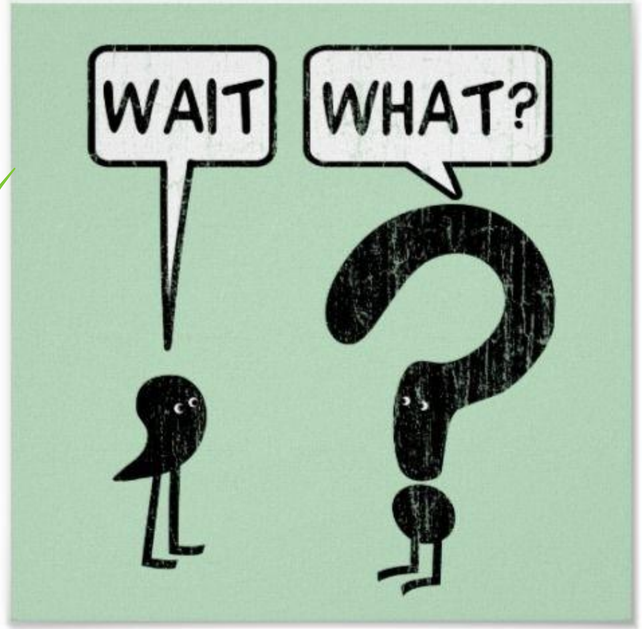


Your turn!

- Aufgaben:
  1. Kreiere einen eigenen…

     …Line Plot
     …Bar Plot
     …Pie Plot
     …Scatter Plot
     …Box Plot

     Nutze dabei verschiedene Extras.
  2. Verpacke alle deine hübschen Plots in einen noch hübscheren großen Plot.
  3. Vorbereitung für nächste Woche: PsychoPy installieren (siehe nächste Folie).



PsychoPy

- Python Toolbox für PsychologInnen
- Hilfreiche Add-ons zum Erstellen von Experimenten in zwei Varianten:
  - als GUI-Builder mit grafischem Interface
  - in Skriptform
- vollständig open-source und mit Linux, Windows und mit Mac kompatibel

PsychoPy

Website

Download & Installation